

Guia Completo de SQL DML no MySQL

Introdução ao SQL DML

SQL DML (Data Manipulation Language), ou Linguagem de Manipulação de Dados, é um subconjunto da linguagem SQL (Structured Query Language) responsável por gerenciar e manipular os dados armazenados em um banco de dados relacional. Os comandos DML permitem aos usuários inserir novos registros, atualizar informações existentes, excluir dados e consultar registros de forma eficiente. É fundamental para a interação diária com qualquer sistema de banco de dados, sendo a base para a maioria das operações de aplicação que envolvem dados.

Em contraste com a **DDL** (Data Definition Language), que se concentra na definição e modificação da estrutura do banco de dados (como tabelas e índices), a DML atua diretamente sobre o conteúdo dessas estruturas, permitindo que os dados sejam dinamicamente gerenciados ².

Comandos Essenciais do SQL DML

Os quatro comandos DML mais importantes e frequentemente utilizados são `INSERT`, `UPDATE`, `DELETE` e `SELECT`. Cada um possui uma função específica na manipulação de dados.

1. INSERT

O comando `INSERT` é utilizado para adicionar novas linhas (registros) a uma tabela. Existem duas formas principais de utilizá-lo:

Sintaxe:

SQL

```
INSERT INTO nome_da_tabela (coluna1, coluna2, coluna3, ...)
VALUES (valor1, valor2, valor3, ...);
```

Ou, se você estiver inserindo valores para todas as colunas na ordem em que foram definidas na tabela:

SQL

```
INSERT INTO nome_da_tabela
VALUES (valor1, valor2, valor3, ...);
```

Exemplo:

Para inserir um novo produto na tabela `produtos` :

SQL

```
INSERT INTO produtos (nome, preco)
VALUES ('Notebook', 1000);
```

É possível também inserir dados de outra tabela utilizando `INSERT INTO SELECT` 2 .

2. UPDATE

O comando `UPDATE` é empregado para modificar dados existentes em uma ou mais linhas de uma tabela. É crucial utilizar a cláusula `WHERE` com `UPDATE` para especificar quais registros devem ser alterados, evitando a modificação acidental de toda a tabela 2 .

Sintaxe:

SQL

```
UPDATE nome_da_tabela
SET coluna1 = novo_valor1, coluna2 = novo_valor2, ...
WHERE condicao;
```

Exemplo:

Para aumentar o salário de todos os funcionários com cargo de 'Gerente' em 10%:

SQL

```
UPDATE funcionarios
SET salario = salario * 1.1
WHERE cargo = 'Gerente';
```

3. DELETE

O comando `DELETE` remove uma ou mais linhas de uma tabela. Assim como o `UPDATE` , a cláusula `WHERE` é essencial para evitar a exclusão de todos os registros da tabela 2 .

Sintaxe:

SQL

```
DELETE FROM nome_da_tabela
WHERE condicao;
```

Exemplo:

Para excluir todos os pedidos realizados antes de uma determinada data:

SQL

```
DELETE FROM pedidos
WHERE data_pedido < '2023-01-01';
```

4. SELECT

O comando `SELECT` é o mais utilizado em SQL DML, permitindo recuperar dados de uma ou mais tabelas. Ele oferece grande flexibilidade para filtrar, ordenar e combinar informações ¹.

Sintaxe Básica:

SQL

```
SELECT coluna1, coluna2, ...
FROM nome_da_tabela
WHERE condicao
ORDER BY coluna ASC/DESC;
```

Para selecionar todas as colunas, utiliza-se o asterisco (*):

SQL

```
SELECT *
FROM nome_da_tabela;
```

Exemplo:

Para selecionar o nome e a idade de clientes da cidade de 'São Paulo':

SQL

```
SELECT nome, idade
FROM clientes
WHERE cidade = 'São Paulo';
```

Operadores e Cláusulas do SELECT

O comando `SELECT` pode ser combinado com diversas cláusulas e operadores para refinar as consultas e obter resultados mais específicos.

Cláusula WHERE

A cláusula `WHERE` é usada para filtrar registros com base em uma condição especificada. Ela pode ser combinada com operadores lógicos e de comparação.

Operadores Aritméticos:

Símbolo	Operação	Tipos de Dados Suportados
<code>+</code>	Adição	BigInt, Int, MediumInt, SmallInt, TinyInt, Float, Double, Real, Numeric e Decimal
<code>-</code>	Subtração	BigInt, Int, MediumInt, SmallInt, TinyInt, Float, Double, Real, Numeric e Decimal
<code>/</code>	Divisão	BigInt, Int, MediumInt, SmallInt, TinyInt, Float, Double, Real, Numeric e Decimal
<code>*</code>	Multiplicação	BigInt, Int, MediumInt, SmallInt, TinyInt, Float, Double, Real, Numeric e Decimal
<code>%</code> ou <code>MOD</code>	Módulo	BigInt, Int, MediumInt, SmallInt, TinyInt, Float, Double, Real, Numeric e Decimal

Operadores Lógicos:

Símbolo	Operação
<code>AND</code>	e
<code>OR</code>	ou
<code>NOT</code>	Negação

Operadores de Comparação:

Símbolo	Operação
=	Igual
>	Maior
<	Menor
>=	Maior e Igual
<=	Menor e Igual
<>	Diferente
IS NULL	Igual a nulo
IS NOT NULL	Diferente de nulo
IN	Igual a valores de uma lista
NOT IN	Diferente de valores de uma lista

Exemplo com múltiplos critérios:

SQL

```
SELECT *
FROM pedido
WHERE valor > 1000 AND data < '20040317';
```

Cláusula ORDER BY

A cláusula `ORDER BY` é usada para classificar o conjunto de resultados de uma consulta em ordem crescente (`ASC`) ou decrescente (`DESC`) com base em uma ou mais colunas.

Exemplo:

SQL

```
SELECT codigo, codcli, data, valor
FROM pedido
ORDER BY valor ASC; -- Ordena pelo valor em ordem crescente
```

Boas Práticas e Otimização

Para garantir a segurança, performance e manutenibilidade do seu banco de dados, é fundamental seguir algumas boas práticas ao utilizar SQL DML 3 .

Segurança e Prevenção de Erros

- **Sempre use WHERE em UPDATE e DELETE** : A omissão da cláusula WHERE em comandos UPDATE ou DELETE resultará na modificação ou exclusão de *todos* os registros da tabela, o que pode causar perda irreversível de dados. Sempre teste suas condições WHERE com um SELECT antes de executar comandos de modificação 2 .
- **Evite SELECT *** : Embora conveniente, SELECT * recupera todas as colunas de uma tabela, mesmo as que não são necessárias. Isso pode levar a um maior tráfego de rede, uso desnecessário de memória e dificultar otimizações. Especifique sempre as colunas que você realmente precisa 3 .
- **Use LIMIT com sabedoria**: Em consultas que podem retornar um grande número de registros, use LIMIT para restringir o número de linhas retornadas. Isso é especialmente útil para depuração ou para exibir resultados paginados 3 .

Performance e Índices

- **Indexe suas cláusulas WHERE** : Índices são estruturas especiais que aceleram a recuperação de dados. Colunas frequentemente usadas na cláusula WHERE (para filtragem) ou ORDER BY (para ordenação) são boas candidatas para indexação. No entanto, o excesso de índices pode diminuir a performance de operações INSERT , UPDATE e DELETE , pois o banco de dados precisa manter os índices atualizados 3 .
- **Índices Compostos**: Para consultas que filtram por múltiplas colunas, um índice composto (em várias colunas) pode ser mais eficiente do que índices individuais. A ordem das colunas no índice composto é importante; coloque a coluna mais seletiva (aquela que filtra mais registros) primeiro 3 .
- **EXISTS vs IN** : Em subconsultas, EXISTS geralmente é mais performático que IN , especialmente para grandes conjuntos de dados, pois EXISTS para de procurar assim que encontra uma correspondência, enquanto IN avalia a subconsulta inteira 3 .

Técnicas Avançadas

Window Functions

As *Window Functions* (Funções de Janela) permitem realizar cálculos em um conjunto de linhas relacionadas à linha atual, sem a necessidade de agrupar os resultados. Elas são extremamente poderosas para cálculos como totais acumulados, médias móveis e ranqueamento 3 .

Exemplos:

- **Total Acumulado:** `SUM(total) OVER (ORDER BY created_at)`
- **Ranqueamento:** `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC)`

Transações

Transações são sequências de operações SQL que são executadas como uma única unidade lógica de trabalho. Elas garantem a **atomicidade** (tudo ou nada), **consistência** (mantém a integridade dos dados), **isolamento** (operações independentes) e **durabilidade** (mudanças permanentes) - conhecidas como propriedades ACID. Os comandos `BEGIN` , `COMMIT` e `ROLLBACK` são usados para gerenciar transações 3 .

- `BEGIN` : Inicia uma transação.
- `COMMIT` : Salva todas as mudanças feitas na transação.
- `ROLLBACK` : Desfaz todas as mudanças feitas na transação.

É crucial manter as transações curtas para evitar bloqueios no banco de dados, que podem impactar a performance de outras operações 3 .

Diferença entre DELETE e TRUNCATE

Embora ambos os comandos possam ser usados para remover dados de uma tabela, `DELETE` e `TRUNCATE` operam de maneiras fundamentalmente diferentes 2 .

Característica	DELETE	TRUNCATE
Tipo	DML (Data Manipulation Language)	DDL (Data Definition Language)
Uso WHERE	Sim, permite filtrar registros	Não, remove todas as linhas
Velocidade	Mais lento (remove linha por linha)	Mais rápido (reinicia a tabela)
Logs	Gera logs para cada linha excluída	Não gera logs de linha (apenas de operação)
Rollback	Sim, pode ser revertido com <code>ROLLBACK</code>	Não, não pode ser revertido
Contador	Mantém o contador de auto-incremento	Reseta o contador de auto-incremento

`DELETE` é ideal para remover registros específicos ou um subconjunto de dados, enquanto `TRUNCATE` é mais adequado para esvaziar completamente uma tabela quando todos os dados são desnecessários e a performance é crítica.

Conclusão

O SQL DML é a espinha dorsal da interação com bancos de dados relacionais, permitindo a manipulação eficaz dos dados. Dominar os comandos `INSERT`, `UPDATE`, `DELETE` e `SELECT`, juntamente com as boas práticas de segurança e otimização, é essencial para qualquer profissional que trabalhe com bancos de dados. A compreensão de conceitos avançados como *Window Functions* e o gerenciamento de transações eleva ainda mais a capacidade de construir sistemas robustos e performáticos.

Referências

- [1] Secoda. Mastering SQL: How to Use SELECT, INSERT, UPDATE, and DELETE Commands. Disponível em:
- [2] AlexOmegapy. MySQL Data Manipulation: Using INSERT, UPDATE, and DELETE Commands. Disponível em:
- [3] Tan, Darren. SQL Best Practices: Hard-Learned Lessons from my years as a Software Engineer. Medium. Disponível em: